01 Virtualization vs Containerization

Virtualization

Virtualization is the creation of software-based (virtual) representations of physical hardware resources — including servers, storage, and networks — allowing multiple isolated environments to run on a single physical machine simultaneously.

Structure

Physical host => Hypervisor => VM (has its own OS/Apps) => Isolation

Benefits of it

1. Multiple VMs/Apps on 1 physical server
2. High server utilization => Hardware cost reduction
3. Provisioning new VM takes minutes
4. Easy to replicate (snapshot and clone)
5. Full isolation between workloads (HW level isolation)
6. Disaster recovery & business continuity
7. Mature, battle-tested technology
8. Best for legacy apps

Cons

1. Each VM Full OS: High Storage usage/slow startup/high memory usage
2. Virtualization layer performance overhead
3. Not ideal for microservices
4. Licensing costs can be significant

Hypervisor types

1. Type 1: Bare metal
   - H/W =========> Hypervisor =====> VMs
   - Best performance/stability/security (e.g., ESXi, KVM, Xen, Hyper-V Server)
2. Type 2: Hosted
   - H/W => Host OS => Hypervisor(as app) => VMs
   - Easier to install/configure (e.g., Virtual Box, VMware Workstation, Parallels)

Containerization

A Lighter, Faster Way to Package & Run Applications
e.g., Docker, containerd, CRI-O, Podman

Container:

A container is a lightweight, standalone, executable package that includes EVERYTHING needed to run an application: code, runtime, libraries, environment variables, and config files — but shares the HOST OS kernel

Structure of container

H/W => Host OS (shared Kernel) => Container runtime (e.g., docker) => Minimal OS filesystem => App runtime => App dependencies => Your application code

and it has a Thin writable layer (runtime changes)

Benefits

1. Lightweight (shares HOST OS Kernel)
2. Portable
3. Fast starting time (ms)
4. Isolated (but only process-level) (Also kernel vulnerabilities can expose every container)
5. Near-native performance
6. Can have hundreds/thousands container per host
7. Best for microservices

How it works?

It uses 2 Linux kernel features

1. Namespaces
2. cgroups

Namespaces

- Linux kernel had this feature long time ago
- Used to provide isolation & security for testing purposes
- You can give a namespace its own POV for the system
  Processes in a namespace can only see each other not other namespaces
  - NET: You can isolate it from using network at all (or have its own network)

    - MNT: its own filesystem
    - PID: It can run with PID 1 (It thinks it's the only thing running)
    - UTS: its own hostname
    - IPC: signals between namespaces
    - User: (let a process be root on its own namespace)
- By default, your Linux uses a default namespace to run all processes

cgroups

Limit/control usage of

1. CPU
2. Memory
3. I/O
4. Network (bandwidth & priority)
5. Device access
6. PID (number of processes)

Container Image Layering

1. Each layer is read-only and cached
2. Layers is reused to save disk space
3. Fast builds (only changed layers are rebuilt/pulled)
4. Containers add thin writable layer on top

Node example

Layer 1: FROM node:18-alpine (Base) — 48.3 MB — sha256: 1e2b..

Layer 2: WORKDIR /app — 0 MB — sha256: 9f4a...

Layer 3: COPY package.json — 0.02 MB — sha256: 2c8d...

Layer 4: RUN npm install — 45.3 MB — sha256: 7b19...

Layer 5: COPY . . — 2.1 MB — sha256: a3f2...

When a container makes changes in an old layer, docker copies that file and put in the writable layer and then write to it. Original layers are read-only

- This has performance overhead in write-intensive apps (frequent update existing files)
    - You can otherwise use volumes for near native I/O performance

Efficient disk usgae

Disk space of 100 containers from the same image = Initial image size + 100 * each writable layer of each container (only)

$$Size_{total} = Size_{image} + \sum_{i=1}^{100} w_i$$

02 Docker architecture - Dockerfile commands

Docker architecture

1. Docker client (CLI tool)
   (Tool to Interact with docker)
   ↓↓
   REST API
2. Docker Daemon (dockerd)
   (Manage images, containers, volumes, networks) it uses containerd
   ↓↓
   Push/Pull
3. Docker registry
   (Remote storage for docker images)

Dockerfile

Text file with step-by-step instructions to build a docker image
Each structure creates a new layer

Dockerfile Instructions

FROM

Base image to start from
e.g., `FROM node:18-alpine`
e.g., `FROM scratch`
e.g., `FROM registry/repo/image:tag`
e.g., `FROM image@sha256:9bca6.....`

WORKDIR

Makes directory if not exists and also `cd` to it (Sets the working directory)
e.g., `WORKDIR /app`
You can say `WORKDIR child_dir` to change working directory to a child directory of `/app`

COPY

Copy files into the image e.g., `COPY . .`
e.g., `COPY file1 file2 destination/`
e.g., `COPY file* destination/`
e.g., `COPY ["file with space.py", "/app"]`

ADD

It's like `COPY` but with additional features like:

- Extracting `.tar` files
- fetch files from remote URL

RUN

Execute commands at build time

- The state of the container after a `RUN` command will be committed to the container image.
- A Dockerfile can have many `RUN` steps that layer on top of one another to build the image.
  e.g., `RUN pip install -r requirements.txt`
- Every `RUN` instruction creates a new layer. To keep images small, we chain commands.
  e.g., `RUN command && command`

1. Shell form
   e.g., `RUN npm install`
   Shell form has environment variables/ piping / command chaining by default
2. Exec form
   e.g., `RUN ["executable", "argument"]`
   does not involve a shell

EXPOSE

Document that the port(s) the app uses e.g., `EXPOSE 3000` (only a comment)

ENV

Global on all layers
Set environment variables

1. set one variable `ENV KEY value` or `ENV KEY=value`
2. set many variables in one line `ENV KEY=value KEY2=value2 KEY3=value3`

LABEL

Metadata about the image `LABEL key="value"`
e.g., `LABEL maintainer="Ali"`

ARG

A variable used inside the dockerfile itself
arguments can be passed while building or has default values
e.g.,

```
ARG username=ali
USER $username
```

ENTRYPOINT

Main executable for the container e.g., `ENTRYPOINT ["./start.sh"]`

You can combine ENTRYPOINT with CMD
ENTRYPOINT for the main exe and CMD for the args

CMD

Default command when container starts e.g., `CMD ["node", "server.js]`
A Dockerfile will only use the final `CMD` defined.
The `CMD` can be overridden when starting a container with `docker run $image $other_command`.

USER

sets the user for executing next commands

SHELL

Set the used shell for subsequent RUN, CMD, ENTRYPOINT instructions
Default for windows is `cmd` and Linux `sh`
e.g., `["/bin/bash", "-c"]`

HEALTHCHECK

the command to run to check if the container is healthy

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=20s --retries=3 \
    CMD wget -qO- http://127.0.0.1:8000/api/health || exit 1
```

Example

```
# Use official Node.js Alpine image
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy package files first (layer caching!)
COPY package*.json ./
RUN npm ci --only=production

# Copy rest of app code
```

```
COPY . .

# Expose port and set entrypoint
EXPOSE 3000

CMD ["node", "app.js"]
```

.dockerignore

A file like .gitignore and is used to ignore files from being copied by a COPY instruction

03 Images - containers - debugging commands

Images

A Docker Image is a READ-ONLY, layered template used to create containers. Think of it as a class— containers are instances of an image.

Image Tags

Naming: `user/repo_name:tag`

Change tag

This creates a new name to the same image (you should remove old image)
`docker tag <old name> <new name>`

Image Id

Each image has unique SHA256 hash

Image manifest

JSON file describing layers and their checksums. Ensures integrity

Build image

Executes instructions in dockerfile to create an image (current directory is sent to dockerd)
`docker build -t name:tag .`

Create image out of a container

`docker commit <container> <image>` not considered best practice

Image registries

Repositories of docker images: e.g., docker hub (public), AWS ECR, GCR (Google), GitHub Container Registry, Harbor (self-hosted)

Pull image

`docker pull user/repo_name:tag`
Pulls from docker hub by default

Push images

`docker push user/repo_name:tag`

List current local images

`docker images`

Remove image

Remove image: `docker rmi image_name`
Remove all unused/dangling images: `docker image prune`

Container

Run container from image

`docker run <image_name>`
`--name` specify a name
`-d` : detached (in background)
`-p host_port:container_port` : Map ports
`-e key=value` add env variables on the fly
`-v` attach volume/bind mount
`--add-host name:ip` run and add an entry in hosts file inside that container
`--memory="512m" --cpus="1.5"`
`--restart` restart policy when failure
options:
`on-failure` : when container exits with error (can specify max retries)
`always` : Always restart if it stops (if stopped manually, it will start again when daemon restarts)
`unless-stopped` : it always restarts unless it's stopped manually

List containers

Running: `docker ps`
All: `docker ps -a`

Stop

`docker stop <container-name-or-id>`

Remove

`docker rm <container-name-or-id>`

Copy files in a container

`docker cp <source> cid:<destination>`

`Ctrl-p` followed by `Ctrl-q` is the default key sequence used to **detach from an interactive Docker container** without stopping it.

Debugging & Logs

Shows real-time logs

```
docker logs -f <c_id>
```

Run commands in the container

```
docker exec -it <c_id> bash
```

View container details

```
docker inspect <c_id>
```

CPU/Memory usage

```
docker stats
```

See processes

```
docker top <c_id>
```

04 Volumes

Containers are EPHEMERAL — when they removed, all data inside is LOST. Volumes solve this by mounting storage that persists outside the container lifecycle.

Named volumes

Managed by docker and stored in `/var/lib/docker/volumes/`
`-v volume_name : /persisted/path/inside/container`

`docker run -v mydata:/data nginx`

- Use in production

Bind mounts

Mount any host directory to use inside
`-v /host/path : /mount/location`

`docker run -v $(pwd):/app node`

- Use in development for testing purposes

tmpfs

Stored in host's memory only. Best for sensitive temp data

`docker run --tmpfs /tmp nginx`

05 Networking

Types

Bridge

Default if not network specified.

When you create a network, its driver is bridge
Provides DNS resolution
Containers on the same bridge network can call each other by container names

Feature	Default Bridge Network	User-Defined Bridges Network
Communication	Containers can only communicate by IP address (unless using the legacy `--link` flag).	Containers can communicate by name or alias, which is better for service discovery.
Isolation	All containers started without a specified network connect to it by default, increasing communication risk between unrelated services.	Provides scoped, better isolation; only containers explicitly connected to the network can communicate.
Flexibility	Containers cannot be connected or disconnected on the fly; requires stopping and re-creating the container.	Containers can be connected or disconnected from networks on the fly during their lifetime.
Configuration	Configuration is done outside of Docker (via `daemon.json`) and requires a Docker restart, applying to all containers on it.	Created and configured with the `docker network create` command, allowing specific settings (subnet, gateway, etc.) for each network.

Host

`docker run --network host`

- Shares hostname of Host OS
- Shares networks of the host
- Does not require port mapping (if 2 use port 80, the second one will **fail to start**)

None

Disconnected from any network
`--network none`

Macvlan

Assigns a unique MAC address to a container, making it appear as a physical device on the network. This is ideal for legacy apps that need to be on a specific VLAN.

Overlay

in multi-host docker environments

Create network

`docker network create <network name>`
Options:
`--subnet 10.0.0.0/16` change default subnet
`--internal` disconnected from internet

Assign network to container

`docker network connect/disconnect <network> <container>`

Commands

- `docker network inspect <name>`
  Crucial for seeing which containers are currently attached and what their internal IP addresses are.
- **`docker network prune`** : Removes all unused networks to keep your `ip addr` output clean.

06 Docker Compose

Docker compose is used when I want to have many containers with their own networking and volumes (e.g., Microservices) and run/stop them as a single unit.

Docker compose was a python module but now it's integrated directly into Docker

docker-compose.yml

Describes what you need to do with docker compose

> **Note**
>
> `.yml` files are like `.json` but without braces
> key: value
> key:
>
> > key: value
> > key: value

`docker-compose.yml` main components

1. version (optional)

```
version: "3.7"
```

2. services (can have multiple services)

For each service in `services` section:

- `image` or `build`

```
# Build image from file
services:
  service_1:
    build: .
```

```
# Use image
services:
  service_1:
    image: "my_image:latest"
```

- `command` : Command runs when the container starts

- `container_name`
- `ports` (target, published) exposed ports

```
services:
  service_1:
    ports:
      - target: 80
        published: 8080
```

OR

```
ports:
    - "8080:80"
```

- `environment` : env variables hardcoded or from .env file

```
services:
  service1:
    environment:
      - DEBUG=true
      - DB_PASS=${DB_PASS}
```

```
# Read env from custom file
services:
  service1:
    env_file:
      - ./file.env
```

- `depends_on` (if one service depends on another)
  Conditions available: `service_started` , `service_healthy` (require a health check), `service_completed_successfully`

```
services:
  service1:
    depends_on:
      service2:
        condition: service_healthy
        restart: true # restart service1 if service2 is updated
```

- `networks`
  Used networks in that service

```
services:
  service_1:
```

```
    networks:
      - mynet
```

- `volumes`
  Used volumes in that service (type, source, target)
  type: volume or bind

```
services:
  my_service_1:
    volumes:
      - type: volume
        source:
        target:
```

- `healthcheck` : the command to run to check if the container is healthy

```
services:
  service1:
    healthcheck:
        test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d
$${POSTGRES_DB}"]
        interval: 10s
        retries: 5
        start_period: 30s
        timeout: 10s
```

- `restart` restart policy when failure
- `logging` : stdout/stderr logging of the container

```
logging:
  driver: json-file
  options:
    max-size: 20m
    max-file: 3
```

- `deploy` : specify resource limits

```
deploy:
  resources: {limits: {memory: 512M}}
```

3. networks

You define networks inside it

```
networks:
    mynet:
        #internal: true
```

4. volumes

```
volumes:
    myvol:
```

Full example

```
version: "3.7"
services:
    service_1:
        build: .
        command: python app.py
        ports:
            - target: 5000
              published: 5000
        networks:
              - mynet
        volumes:
            - type: volume
            source: myvol
            target: /data
    service_2:
        image: "python:latest"
        command: python app.py
        ports:
            - target: 5000
              published: 5000
        networks:
              - mynet
        volumes:
            - vol1:/data
            - /dir:/app/data
volumes:
    myvol:
networks:
    mynet:
```

Run the docker compose

`docker compose up`
`-d` detached
`--build` rebuild
You find the 2 containers running.

Naming

If no container name is specified, their names is
`directory-image1_name-1`,
`directory-image2_name-1`

Pinging

They can ping each other by container name or by service name

---

Run command inside a service

1. You can `exec` with container's hash
2. You can use service_name
   `docker-compose exec service_name command`

Check services running in docker compose

```
docker-compose ps
```

Stop the docker compose

`docker compose stop`

Down

`docker compose down`
This deletes the containers/networks not volumes not images
`-v` : removes volumes too

07 Best practices - Orchestration

Multi-stage builds

Multi-stage builds use multiple FROM statements. Each stage can have a different base image. You copy only what you need from earlier stages — keeping the FINAL image tiny and secure.

- Builder stage contains all tools required to build the source code
- Production stage: as tiny as possible (only code runtime)
- Less disk space & more secure

```
# Stage 1: BUILD
FROM golang:1.21 AS builder

WORKDIR /app

COPY go.* .

RUN go mod download

COPY . .

RUN CGO_ENABLED=0 go build -o app

# Stage 2: PRODUCTION
FROM scratch # empty base!

COPY --from=builder /app/app /usr/local/bin/app

CMD ["app"]
```

Common pipeline

1. Code committed to Github
2. Build a new version of docker image
3. Run tests inside a test container
4. Scan image for vulnerabilities with (e.g., Trivy)
5. Push new version to registry (e.g., dockerhub)
6. Pull & deploy in staging/production

Container Orchestration

When you have hundreds or thousands of containers across many servers, you need an orchestration platform to manage deployment, scaling, networking, health checks, and updates automatically.
e.g., Docker Swarm, Kubernetes, AWS ECS / Fargate

e.g.,

1. Restart when crash
2. Run new containers when traffic is more
3. Rolling updates without downtime
4. Load balancing
5. Service dicovery (App A finds App B)
6. Secrets & config management at scale

Best Practices

Dockerfile

1. Use small base images
2. Use multi-stage builds
3. **Layer Ordering:** Always put the most "stable" instructions at the top.
   - Copy package files before source code (to be cached in future builds)
4. use COPY --link (independent of previous layers)
5. Pin exact versions in FROM (don't use latest)
6. use .dockerignore (.git/, .env, node_modules/, bin/, obj/)

Security
7. Don't run as root
8. Scan images
9. No hardcode secrets in images
10. Use Read-only filesystems when possible
11. Run with `--read-only`. This prevents attackers from downloading and running malware inside your container.
12. Regularly update base images
Images
13. Tag images with git SHA or SemVer (Semantic Versioning) is a 3-part versioning scheme (MAJOR.MINOR.PATCH)
14. Use private registries if needed
15. Clean up unused images
16. Use health checks in production
Runtime
17. Set CPU, memory limits
18. use restart: unless-stopped (to restart container if terminates unexpectedly)
19. Use environment variables for config

20. Log to stdout/stderr not files (files are deleted with container but logs persists)
21. Ensure your application handles `SIGTERM` (in order to do a graceful shutdown)